Яппаров Эмир DS

ИТ-стартап (Solana): отчёт по практическим заданиям 1-6

Сводка выполнения

Работа подготовлена по курсу ИТ-стартап Итоговый проект 1 для группы Data Science 3 курса. Преподаватель: Иманкулова Алия Жангельдиновна. В проект собраны материалы по шести обязательным практическим заданиям: вход в Web3, разбор транзакций Transaction Detective, CLI-программа на Rust, первая программа Solana, счётчик на Anchor с клиентской проверкой и профиль пользователя на PDA.

Важное исправление к прежней отправке от 1 октября: в том файле за перевод студента была ошибочно выдана чужая транзакция Devnet, а вывод локального симулятора был описан как работа в сети. В настоящем отчёте все результаты разделены честно: где запросы шли в живой Devnet RPC, где разбирались публичные транзакции других участников и где выполнялись локальные тесты программ.

Задание 1. Кошелёк, тестовый SOL и первая транзакция

Для работы созданы два настоящих Ed25519-кошелька Solana: основной адрес студента 2SzJ2PSuhnQsenfvHK4DhQHibjVL2d68fwdHAMwxJ7pL и адрес получателя AK3KTtGPrAXY769tLueYsFucMgRs7ZYpGHsub6cA6ZF1. Приватные ключи сохранены только локально с правами 600 и не включены ни в отчёт, ни в архив.

Через публичный узел https://api.devnet.solana.com проверен баланс на слоте 510169097: на счёте 0 lamports. Автоматические запросы requestAirdrop к публичному RPC и через solfaucet.com вернули ошибки Internal error и код 429 о лимите крана, а веб-краны faucet.solana.com и devnetfaucet.org требуют ручного прохождения Cloudflare или входа через GitHub в браузере студента.

Из-за нулевого баланса перевод в публичном Devnet пока не отправлен. Скрипт отправки devnet_work.py полностью готов: как только на адрес студента поступит тестовый SOL, он подпишет и отправит реальный SystemProgram::Transfer. Дополнительно тот же перевод проверен в локальной виртуальной машине LiteSVM: после пополнения тестового счёта на 2 SOL перевод 100000000 lamports завершился со статусом success, баланс получателя стал равен 100000000 lamports.

Задание 2. Transaction Detective: разбор реальных транзакций

Для второго задания через метод getTransaction с параметром jsonParsed получены две настоящие подтверждённые транзакции публичной сети Devnet. Они принадлежат сторонним участникам сети и используются только как объект анализа.

Первая транзакция: XV7fffZmLkGh8Vr4AvVdh9PcZkqtFniScaro8c5uYBoo8QGgmi1HMFUs3ubGTju XgDdtEMdKnXgHRpLYhHbDdVU в слоте 510170106. Плательщик комиссии: 6a1wxRdkWZKPHqSJvEEwcd9KywCEtrSnmswHDhNsBNqd, списано 35000 lamports, ошибка выполнения отсутствует. Внутри пакета 6 инструкций: две инструкции ComputeBudget111111111111111111111111111111 задают лимит вычислений и приоритетную комиссию, программа MemoSq4gqABAXKb96qnH8TysNcWxMyWCqXgDLGmfcHr записывает строку метаданных сделки, два вызова ATokenGPvbdGVxr1b2hvZbsiqW5xWH25efTNsLJA8knL выполняют createIdempotent для токен-аккаунтов, после чего TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA переводит 1000000000 базовых единиц токена mint 4wMp7FQJrSwHXJ92kKRd9X2LQE1Tw7L4beT7fBLgjY4y.

Вторая транзакция: 32yNDQx9iRVsxKsFj2BBpaWn1BpiW4RhytN8gmX7LyV6y2gLExjRtHbeXLLQXr DSHVWW7qJCdvuuUgvqiaTWs96h в слоте 510170105. Плательщик комиссии: AVm1wwtnLtsdJSpgz8S2SfC4JDepipEwBpLYuHgQ25Yb, комиссия 15700 lamports. После двух инструкций Compute Budget и записи в SPL Memo создаётся связанный токен-аккаунт DFJbHB9qhxWm7QeZ5DS478j6YHBZHbzJaTG7wk5FRCCh для mint

Aj1qyqJRvi7VFdCriNvLiiCvjkLW3pTRgFCKSyGtAbsk, выполняется перевод 69511888 единиц токена и вызывается пользовательская программа Bz4AoAXL8vkTSFPqR52GwNxFixAbKPJ3VShhgoS2eV2G. Поскольку её IDL не опубликован в RPC-ответе, данные инструкции оставлены в исходном Base58 виде без домыслов о названии метода.

Разбор подтверждает ключевой принцип архитектуры Solana: исполняемый код программ отделён от аккаунтов данных, права подписи и изменения задаются флагами signer и writable в заголовке сообщения, а перевод SPL-токена меняет поля токен-аккаунтов, но не является системным переводом SOL.

Задание 3. CLI-программа на Rust для разработчика Solana

В каталоге rust-cli написана и собрана консольная утилита solana-lab-cli на Rust. В отличие от старого макета она не печатает зашитые в код хеши, а проверяет длину адресов и сигнатур по алфавиту Base58 и обращается к настоящему JSON-RPC узлу https://api.devnet.solana.com.

В программе использованы структуры, перечисления команд и ошибок, сопоставление с образцом match, обработка ошибок через Result и модульные тесты. Команда cargo test проходит все 5 проверок: адрес System Program декодируется ровно в 32 байта, строка с запрещённым символом l отклоняется, короткая сигнатура отклоняется, запрос баланса для валидного ключа проходит.

```
cargo test --manifest-path rust-cli/Cargo.toml
running 5 tests
test tests::invalid_chars ... ok
test tests::short_signature ... ok
test tests::system_address ... ok
test tests::usage ... ok
test tests::valid_balance ... ok

cargo run --manifest-path rust-cli/Cargo.toml -- balance 2SzJ2PSuhnQsenfvHK4DhQHibjVL2d68fwdHAMwxJ7pL
{"jsonrpc":"2.0","result":{"context":{"apiVersion":"4.4.0-beta.0","slot":510169500},"value":0},"id":1}
```

Задание 4. Первая программа Solana

В каталоге programs/first-program реализована нативная программа на крейте solana-program 2.3.0 с точкой входа entrypoint!(process_instruction). Программа принимает байтовый буфер инструкции: при передаче байта 0 она записывает приветствие в журнал выполнения через макрос msg! и возвращает Ok(()), а для любых других входных данных возвращает ошибку InvalidInstructionData.

Модульные тесты проверяют успешный вызов и отказ на неизвестной инструкции. Для реального деплоя в Devnet требуются установленный компилятор cargo-build-sbf и пополненный тестовый кошелёк; на текущем сервере свободно около 108 МБ дискового пространства, поэтому тяжёлый SBF SDK сюда не помещается, а кран Devnet закрыт лимитом. Сама программа и тесты полностью готовы к сборке и выкладке после пополнения кошелька.

```
pub fn process_instruction(_program_id: &Pubkey, _accounts: &[AccountInfo], data: &[u8]) -> ProgramResult {
    match data {
        [0] => { msg!("Hello from Emir's first Solana program!"); Ok(()) },
        _ => Err(solana_program::program_error::ProgramError::InvalidInstructionData),
    }
}
```

Задание 5. Программа счётчика на Anchor и клиентская проверка

В каталоге programs/anchor-counter написан контракт anchor_counter на базе anchor-lang с объявленным идентификатором Fg6PaFpoGXkYsidMpWxTWqkZ8q7kL6K2f3d9c8P4v1A. Аккаунт состояния Counter хранит публичный ключ владельца authority и беззнаковый счётчик count. Реализованы три инструкции: initialize создаёт аккаунт со значением 0, increment

увеличивает счётчик с защитой checked_add, decrement уменьшает значение с проверкой checked_sub.

Ограничение #[account(mut, has_one = authority)] гарантирует, что изменить счётчик может только владелец, подписавший транзакцию. В модульном тесте через точку входа Anchor entry выполнены два инкремента до значения 2, проверен отказ при попытке вызова от чужого подписанта, затем выполнены два декремента до 0 и проверен отказ при попытке уйти в отрицательное значение. Клиентский скрипт clients/solana_client.py вычисляет 8-байтовый дискриминатор инструкции increment (0b12680968ae3b21) и проверяет бинарную сериализацию аккаунта Counter по схеме Borsh.

```
running 2 tests
test test_id ... ok
test tests::counter_client_roundtrip_and_authority_check ... ok
```

Задание 6. Децентрализованный профиль на PDA

В каталоге programs/pda-profile реализована программа pda_profile с идентификатором Fg6PaFpoGXkYsidMpWxTWqkZ8q7kL6K2f3d9c8P4v1B. Профиль пользователя размещается по детерминированному адресу Program Derived Address со связкой семян [b"profile", authority.key().as_ref()] и каноническим bump. Для ключа студента 2SzJ2PSuhnQsenfvHK4DhQHibjVL2d68fwdHAMwxJ7pL клиентом и программой выводится один и тот же адрес PDA 7BwyGhDDWTV6PEjm1CuMZ6jZjtB7oGLT8PV9t7vuGqJL с bump=255.

Инструкция create_profile инициализирует аккаунт UserProfile и сохраняет поля authority, username (до 32 байт), bio (до 160 байт) и bump. Инструкция update_profile проверяет подпись владельца через has_one = authority и совпадение PDA по сохранённому bump. В тесте программы проверены детерминированный вывод адреса, обновление профиля на значения emir_ds и AlmaU 3 course, отказ при подписи чужим ключом и отказ при превышении длины имени пользователя.

```
running 2 tests
test test_id ... ok
test tests::pda_derivation_update_and_owner_check ... ok
```